

LAPORAN PROYEK AKHIR
“Membangun Kompiler Sederhana untuk Bahasa SimpleLog”
TEKNIK KOMPILASI



Dosen Pengampu :

Assoc. Prof. Dr. Susandri, S.Kom, M.Kom

Disusun Oleh :
Kelompok 4

Sandro Natanael	(2455201017)
Rifaldi Sinaga	(2455201043)
Kefraka Ramadana	(2455201047)
Stefani Ana Relina Dian Ocela	(2455201069)
Humaira Adelsi Septa Tindra	(2455201073)
Muhammad Raffi Nugraha	(2455201079)

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS LANCANG KUNING
TAHUN AJARAN 2025/2026

KATA PENGANTAR

Puji dan syukur kami panjatkan kehadirat Tuhan Yang Maha Esa atas segala rahmat, karunia, serta kemudahan yang diberikan-Nya, sehingga kami dapat menyelesaikan proyek akhir sekaligus menyusun laporan yang berjudul "Membangun Kompiler Sederhana untuk Bahasa 'SimpleLog'" ini tepat pada waktunya. Laporan ini disusun sebagai bentuk pertanggungjawaban akademik serta untuk memenuhi salah satu syarat kelulusan dan evaluasi Ujian Akhir Semester pada mata kuliah Teknik Kompilasi di Program Studi Teknik Informatika, Universitas Lancang Kuning.

Penyusunan laporan dan pengembangan perangkat lunak ini merupakan sebuah perjalanan akademis yang sangat berharga bagi kami. Melalui proyek SimpleLog ini, kami tidak hanya dituntut untuk memahami teori-teori abstrak mengenai bagaimana sebuah mesin menerjemahkan bahasa pemrograman tingkat tinggi, tetapi juga diuji untuk mengimplementasikan teori tersebut ke dalam bentuk nyata menggunakan bahasa pemrograman Python. Kami belajar secara langsung bagaimana menstrukturkan alur kompilasi secara utuh, mulai dari proses analisis leksikal dan sintaksis yang cermat, analisis semantik yang presisi, pembuatan *Three Address Code* sebagai representasi perantara, hingga penerapan algoritma optimasi seperti *constant folding* dan *dead code elimination* guna meningkatkan efisiensi hasil kompilasi.

Kami menyadari bahwa keberhasilan penyelesaian proyek dan penyusunan laporan ini tidak terlepas dari bimbingan, dukungan, serta bantuan dari berbagai pihak. Oleh karena itu, pada kesempatan ini kami ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada dosen pengampu mata kuliah Teknik Kompilasi yang telah memberikan ilmu, arahan, serta bimbingan yang sangat berharga selama satu semester ini. Ucapan terima kasih juga kami sampaikan kepada seluruh rekan-rekan anggota Kelompok 4 yang telah mendedikasikan waktu, tenaga, dan pikirannya. Kerja sama tim yang solid dalam memecahkan berbagai kebuntuan logika selama proses penulisan kode adalah kunci utama dari selesainya kompiler SimpleLog ini. Tidak lupa, kami mengapresiasi seluruh pihak yang tidak dapat kami sebutkan satu per satu, yang telah memberikan dukungan moral maupun material selama proses perkuliahan dan pengerjaan proyek.

Akhir kata, kami berharap kompilator sederhana SimpleLog beserta laporan teknis ini dapat memberikan manfaat, inspirasi, dan tambahan wawasan bagi siapa saja yang membacanya. Semoga karya ini dapat menjadi referensi yang berguna, khususnya bagi rekan-rekan mahasiswa Teknik Informatika yang sedang atau akan mendalami dunia teknik kompilasi perangkat lunak.

Pekanbaru, 17 Juni 2026

Kelompok 4

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
DAFTAR GAMBAR	iv
DAFTAR TABEL	v
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	2
1.3 Spesifikasi Bahasa SimpleLog	3
1.4 Batasan Sistem	5
BAB II DESAIN	7
2.1 Arsitektur Sistem	7
2.2 Desain Token	9
2.3 Desain AST (Abstract Syntax Tree)	11
2.4 Diagram Alir Proses Kompilasi	12
BAB III IMPLEMENTASI	13
3.1 Modul Scanner (scanner.py)	13
3.2 Modul Parser (parser.py)	15
3.3 Modul Semantic Analyzer (semantic.py)	17
3.4 Modul ICG (Intermediate Code Generator (icg.py)	19
3.5 Modul Optimasi (optimizer.py)	21
3.6 Modul Error Handling & Integrasi (compiler.py)	23
BAB IV PENGUJIAN	26
4.1 Test Case 1- Program Valid Dasar	26
4.2 Test Case 2 – Error Leksikal	28
4.3 Test Case 3 – Error Sintaks (Multi Error)	29
4.4 Test Case 4 Error Semantik	30
4.5 Test Case 5 – Ekspresi Kompleks & Optimasi	31
4.6 Ringkasan Hasil Pengujian	34
BAB V PENUTUP	36
5.1 Kesimpulan	36

5.2	Saran.....	36
DAFTAR PUSTAKA.....		38

DAFTAR GAMBAR

Gambar 1 Arsitektur pipeline kompiler SimpleLog.....	8
Gambar 2 Grammar BNF lengkap bahasa SimpleLog	9
Gambar 3 Diagram alir proses kompilasi SimpleLog.....	12
Gambar 4 Dispatcher utama pada kelas Scanner	13
Gambar 5 Implementasi prioritas operator melalui struktur rekursif	16
Gambar 6 Validasi identifier pada Semantic Analyzer	17
Gambar 7 Three Address Code hasil ICG (18 instruksi, sebelum optimasi)	19
Gambar 8 TAC setelah optimasi (13 instruksi, penghematan 28%)	21
Gambar 9 Opsi penggunaan CLI compiler.py	24
Gambar 10 Hasil pengujian Test Case 1	26
Gambar 11 Pesan error leksikal beserta penunjuk posisi	28
Gambar 12 Test Case 4 Error Semantik.....	31
Gambar 13 Hasil pengujian Test Case 5	32

DAFTAR TABEL

Tabel 1 Daftar jenis token bahasa SimpleLog	10
Tabel 2 Struktur node AST SimpleLog.....	11
Tabel 3 Spesifikasi Token untuk Keyword dan Simbol pada Kompilator SimpleLog	14
Tabel 4 Kode Sumber Penanganan Karakter dan Pemicu Error Leksikal	14
Tabel 5 Kode Sumber Struktur Data Pohon Sintaksis Abstrak (Abstract Syntax Tree) Bahasa SimpleLog	16
Tabel 6 Kode Sumber Mekanisme Pemulihan Sintaks (Syntax Error Recovery) pada Parser SimpleLog.....	16
Tabel 7 Kode Sumber Validasi Deklarasi Ganda pada Tahap Analisis Semantik.....	18
Tabel 8 Kode Sumber Pemeriksaan Aturan Semantik Penggunaan Variabel Bahasa SimpleLog.....	18
Tabel 9 Kode Sumber Pemformatan String untuk Instruksi Tiga Alamat (Three Address Code).....	19
Tabel 10 Kode Sumber Generasi TAC untuk Ekspresi Logika Optimasi Melompat	20
Tabel 11 Kode Sumber Pemicu Generator Kode Alur Blok Kontrol If-Else	20
Tabel 12 Kode Sumber Penjelajah Optimasi Konstanta pada Kompilator	22
Tabel 13 Kode Sumber Pembersihan Variabel Temporer Tidak Terpakai	22
Tabel 14 Kode Sumber Pengendali Loop Optimasi Tahap Backend Kompilator.....	23
Tabel 15 Kode Sumber Pengendali Sekuensial Antarfase Kompilator Bahasa SimpleLog	24
Tabel 16 Kode Sumber Penanganan dan Pemetaan Posisi Visual Eksplisit Kompilator...	25
Tabel 17 Hasil Pengujian Validasi Fungsional Kompilator Tanpa Error	26
Tabel 18 Riwayat kompilasi lengkap dan penelusuran alur kode antar (TAC) Test Case	126
Tabel 19 Hasil Penelusuran Fase Kompilasi dan Optimasi Kode Pada Skenario Error Leksikal.....	28
Tabel 20 Riwayat Pengujian Skenario Negative Testing dengan Panic-mode Recovery pada Parser.....	30
Tabel 21 Hasil Pengujian Skenario Negative Testing Untuk Validasi Variabel pada Semantik Analyzer	31
Tabel 22 Log Statistik Perbandingan Komparatif Jumlah Instruksi Sebelumnya dan Sesudah Optimasi.....	32

Tabel 23 Hasil Pengujian dan Optimasi Kode Antar (TAC) Pada Skenario Ekspresi Logika Bersarang	32
Tabel 24 Ringkasan Seluruh Hasil Pengujian (5/5 Lolos)	35

BAB I

PENDAHULUAN

1.1 Latar Belakang

Mata kuliah Teknik Kompilasi merupakan salah satu pilar penting dalam keilmuan Teknik Informatika yang menjembatani pemahaman antara bahasa pemrograman tingkat tinggi yang ditulis oleh manusia dengan instruksi tingkat rendah yang dieksekusi oleh perangkat keras. Dalam perkuliahan ini, mahasiswa diajarkan tahapan-tahapan krusial yang dilalui sebuah program sumber sebelum pada akhirnya dapat diproses dan dieksekusi secara valid oleh komputer. Proses transformasi dari kode sumber menjadi bentuk yang dapat dieksekusi ini bukanlah sebuah proses tunggal yang sederhana, melainkan sebuah *pipeline* panjang dan sistematis yang berawal dari analisis leksikal hingga bermuara pada optimasi kode. Mempelajari berbagai algoritma dan teori komputasi di balik setiap tahapan ini memberikan wawasan mendalam tentang bagaimana perangkat lunak penerjemah modern bekerja di belakang layar.

Meskipun demikian, sekadar mempelajari konsep-konsep teoritis kompilasi—seperti *state machine* untuk tokenisasi atau tata bahasa bebas konteks (*context-free grammar*) untuk *parsing*—seringkali menyisakan jarak abstraksi yang besar bagi mahasiswa. Tantangan terbesar dalam pengembangan kompilator adalah mengelola kompleksitas saat mengintegrasikan fase-fase yang berbeda tersebut menjadi sebuah perangkat lunak yang solid dan fungsional. Oleh karena itu, untuk memahami konsep teoritis kompilasi tersebut secara langsung ke dalam bentuk nyata, kelompok kami mengambil inisiatif praktis dengan merancang serta mengimplementasikan sebuah kompiler sederhana untuk sebuah bahasa buatan yang diberi nama SimpleLog. Praktik pembuatan kompilator secara mandiri ini bertindak sebagai medium untuk mentransformasi pemahaman konseptual menjadi keterampilan rekayasa perangkat lunak (*software engineering*) yang terapan.

Dalam proses merancang sebuah bahasa buatan untuk proyek tingkat universitas, penentuan ruang lingkup atau *scope* dari fitur bahasa merupakan keputusan arsitektural yang paling menentukan arah proyek. Untuk itu, SimpleLog didesain sebagai sebuah bahasa pemrograman sederhana yang berfokus sepenuhnya pada operasi logika boolean. Pemilihan paradigma bahasa yang sangat terfokus ini dilakukan melalui pertimbangan yang matang. Bahasa pemrograman ini dipilih secara strategis karena cakupan fiturnya dinilai sudah sangat cukup dan representatif untuk mendemonstrasikan keseluruhan proses dan tahapan kompilasi secara *end-to-end*.

Melalui bahasa SimpleLog ini, setiap komponen inti penyusun kompilator dapat diuji kemampuannya secara utuh. Kelompok kami dapat mendemonstrasikan secara presisi proses *scanning* karakter menjadi token, penguraian (*parsing*) token menjadi *Abstract Syntax Tree*, analisis semantik untuk validasi aturan logika program, pembuatan representasi kode antara (*intermediate code*), optimasi untuk meningkatkan efisiensi eksekusi, serta implementasi sistem penanganan *error* yang baik. Semua pembelajaran fase kompilasi tersebut dapat dicapai tanpa harus terjebak pada kompleksitas berlebihan dari sebuah bahasa pemrograman yang biasanya menuntut implementasi tipe data numerik yang luas atau struktur data kompleks seperti *array* dan *object*. Dengan menjaga batasan fitur

hanya pada operasi logika boolean, proyek ini dapat secara murni berfokus pada kebenaran arsitektur dan alur data di dalam kompilator itu sendiri, menjadikannya sebuah purwarupa akademis yang efisien, terukur, dan tepat sasaran.

1.2 Tujuan

Pelaksanaan proyek akhir pembuatan kompilator untuk bahasa SimpleLog ini dirancang dengan berbagai target capaian yang komprehensif, baik dari sisi akademis maupun praktis rekayasa perangkat lunak. Secara rincian, tujuan utama dari penelitian dan pengembangan sistem ini adalah sebagai berikut:

1. Mengimplementasikan Arsitektur Komponen Inti Kompilator secara Utuh Tujuan pertama dari proyek ini adalah merancang dan membangun keenam komponen esensial dari sebuah kompilator standar, yang meliputi *Scanner* (penganalisis leksikal), *Parser* (penganalisis sintaks), *Semantic Analyzer* (penganalisis semantik), *Intermediate Code Generator* (ICG), modul Optimasi, serta sistem *Error Handling* yang saling terintegrasi. Implementasi ini ditujukan untuk membuktikan bahwa sebuah bahasa pemrograman mandiri (SimpleLog) dapat diproses melalui arsitektur *pipeline* klasik secara berurutan tanpa memutus integritas aliran data antartahap.
2. Menerapkan Teori Komputasi ke dalam Praktik Pemrograman Nyata Proyek ini bertujuan untuk menjembatani kesenjangan antara teori abstrak kompilasi dengan implementasi praktisnya menggunakan bahasa pemrograman Python. Hal ini mencakup penerapan langsung dari konsep tata bahasa bebas konteks (*Context-Free Grammar*) menggunakan notasi BNF, penulisan algoritma *recursive descent parsing* untuk membangun *Abstract Syntax Tree* (AST), serta penciptaan representasi *Three Address Code* (TAC). Selain itu, proyek ini juga bertujuan membuktikan efektivitas algoritma optimasi kode, khususnya teknik *constant folding* dan *dead code elimination*, dalam menyederhanakan instruksi eksekusi.
3. Membangun Sistem Pelaporan Kesalahan (*Error Handling*) yang Komprehensif Mengingat fungsi utama kompilator adalah sebagai alat bantu bagi pemrogram, sistem ini dirancang dengan tujuan menghasilkan umpan balik atau pesan kesalahan (*error messages*) yang sangat informatif pada setiap tahapan kompilasi. Sistem ditargetkan mampu menangkap kesalahan karakter tidak dikenal pada tahap leksikal, mendeteksi struktur tata bahasa yang menyimpang melalui mekanisme *panic-mode recovery* pada tahap sintaksis, hingga menemukan anomali logika seperti variabel yang belum dideklarasikan atau diinisialisasi pada tahap semantik. Umpan balik ini harus secara akurat menunjukkan nomor baris, posisi kolom, serta potongan kode sumber yang bermasalah untuk memudahkan proses *debugging*.
4. Mendemonstrasikan Siklus Kompilasi End-to-End dengan Evaluasi Logika Bersyarat Tujuan akhir dari proyek ini adalah mendemonstrasikan secara visual dan struktural seluruh siklus hidup kode sumber SimpleLog—mulai dari input teks mentah hingga menjadi kode antara yang telah dioptimasi. Demonstrasi ini secara khusus menargetkan keberhasilan penanganan evaluasi logika bersyarat (seperti operasi *short-circuit* pada AND/OR) menggunakan instruksi lompatan bersyarat (*conditional jump*) yang meniru perilaku evaluasi kompilator industri. Dengan demikian, sistem diharapkan dapat memperlihatkan

secara terukur seberapa besar persentase efisiensi dan penghematan instruksi yang berhasil dicapai setelah kode melewati tahap optimasi akhir.

1.3 Spesifikasi Bahasa SimpleLog

SimpleLog dirancang secara khusus sebagai bahasa pemrograman eksperimental berskala kecil (pedagogis) yang menitikberatkan fokus utamanya pada operasi logika boolean dan kontrol percabangan. Bahasa ini lahir dari kebutuhan untuk mendemonstrasikan setiap fase arsitektur dalam kompilator secara mendalam tanpa harus membebani sistem dengan aturan semantik dan leksikal yang terlalu kompleks, seperti manajemen memori tingkat tinggi, alokasi array, atau kalkulasi numerik multi-tipe data. Meskipun memiliki cakupan fitur yang minimalis, SimpleLog mengadopsi karakteristik sintaksis yang ketat mirip dengan rumpun bahasa *C-like*, seperti keharusan menggunakan tanda titik koma (*semicolon*) sebagai terminator akhir instruksi serta penggunaan kurung kurawal untuk mendefinisikan blok cakupan perintah (*scope*).

Secara rinci, komponen dan spesifikasi teknis pembentuk bahasa SimpleLog dijabarkan ke dalam beberapa poin aturan formal berikut:

1. Sistem Tipe Data dan Literal

Bahasa SimpleLog menerapkan sistem tipe data tunggal (*single data type system*), yang berarti sistem hanya mengenali dan mendukung satu jenis tipe data primitif, yaitu boolean. Karena tidak ada tipe data lain seperti *integer*, *float*, atau *string*, kompilator ini tidak membutuhkan modul penanganan konversi tipe data (*type casting*) atau operasi koersi implisit yang rumit. Karakteristik dari sistem tipe data ini meliputi:

- a. **Literal true:** Digunakan untuk merepresentasikan nilai kebenaran logika benar (kondisi aktif atau terpenuhi). Penulisan literal ini bersifat *case-sensitive* menggunakan huruf kecil seluruhnya.
- b. **Literal false:** Digunakan untuk merepresentasikan nilai kebenaran logika salah (kondisi tidak aktif atau tidak terpenuhi). Sama seperti literal teks benar, penulisan literal ini wajib menggunakan huruf kecil seluruhnya.
- c. **Aturan Nilai:** Setiap variabel yang dideklarasikan di dalam program wajib menerima alokasi nilai boolean yang valid dan tidak diperbolehkan berada dalam status tidak terdefinisi saat dievaluasi oleh pernyataan lain.

2. Struktur Leksikal dan Kategori Token

Setiap komponen teks mentah dalam kode sumber SimpleLog akan dipecah menjadi rangkaian token yang bermakna oleh modul pengenalan leksikal (*Scanner*) sebelum dikirim ke tahap penguraian. Spesifikasi leksikal bahasa ini mengelompokkan karakter ke dalam kategori token berikut:

- a. **Kata Kunci (*Keywords*):** Merupakan deretan kata pencadang resmi sistem yang memiliki makna khusus dan tidak dapat digunakan kembali oleh pemrogram sebagai nama variabel. Kata kunci yang terdaftar meliputi boolean (deklarasi tipe data), *true* dan *false* (literal nilai logika), *if* dan *else* (kontrol aliran percabangan), serta *print* (pernyataan keluaran).
- b. **Pengenalan (*Identifiers*):** Merupakan penamaan unik yang ditentukan oleh pemrogram untuk mengidentifikasi sebuah variabel. Aturan sintaksis untuk pengenalan wajib diawali oleh karakter alfabet (a-z, A-Z) atau karakter garis bawah (*underscore* `_`), yang kemudian dapat diikuti oleh kombinasi huruf, angka, atau garis bawah secara dinamis.

- c. **Operator Logika:** Terdiri atas tiga token operator utama yang wajib ditulis menggunakan huruf kapital penuh (*uppercase*), yaitu NOT untuk operasi negasi unary, AND untuk konjungsi biner, dan OR untuk disjungsi biner.
- d. **Simbol Pemisah (*Punctuators*):** Karakter-karakter khusus yang berfungsi menegaskan pembatasan struktural antarelemen kode. Simbol pemisah yang diakui secara sah adalah = (penugasan nilai), ; (terminator akhir baris), (dan) (pembatas ekspresi atau parameter fungsi), serta { dan } (pembatas cakupan blok instruksi).

3. Operator Logika dan Hierarki Prioritas

Untuk menghindari ambiguitas ketika melakukan evaluasi terhadap ekspresi boolean bersarang yang kompleks, SimpleLog menerapkan hierarki prioritas operator yang ketat. Aturan prioritas ini tertanam langsung di dalam struktur tata bahasa bebas konteks (*Context-Free Grammar*) pada bagian Parser, sehingga urutan eksekusi akan berjalan konsisten tanpa memerlukan tabel prioritas eksternal. Urutan prioritas operator dari yang tertinggi hingga terendah didefinisikan sebagai berikut:

- a. **NOT (Negasi — Prioritas Tertinggi):** Merupakan operator unary yang bekerja pada satu operand di sisi kanannya. Operator ini bertugas membalikkan nilai boolean (mengubah true menjadi false dan sebaliknya) dan selalu dievaluasi paling awal.
- b. **AND (Konjungsi — Prioritas Menengah):** Merupakan operator biner yang membutuhkan dua operand. Operasi ini hanya akan menghasilkan nilai true jika kedua operand yang dihubungkan sama-sama bernilai benar.
- c. **OR (Disjungsi — Prioritas Terendah):** Merupakan operator biner tingkat akhir. Operasi ini menghasilkan nilai true jika salah satu atau kedua operand yang dihubungkan memiliki nilai benar.

Apabila pemrogram ingin memanipulasi urutan evaluasi standar ini, penggunaan tanda kurung () diizinkan untuk mengelompokkan sub-ekspresi. Sub-ekspresi yang berada di dalam tanda kurung akan dipaksa untuk diselesaikan terlebih dahulu oleh kompilator.

4. Struktur Sintaksis Pernyataan (Statements)

Alur eksekusi program di dalam SimpleLog dibangun oleh kumpulan pernyataan terstruktur yang ditulis secara berurutan. Ragam pernyataan formal yang didukung meliputi:

- a. **Pernyataan Deklarasi:** Instruksi untuk mendaftarkan variabel baru ke dalam tabel simbol. Sintaksisnya wajib diawali dengan kata kunci boolean, diikuti oleh identifier variabel, operator penugasan =, ekspresi logika penentu, dan diakhiri dengan terminator ;. Contoh: boolean kondisi = true;.
- b. **Pernyataan Penugasan (*Assignment*):** Instruksi untuk memperbarui nilai dari variabel yang telah dideklarasikan sebelumnya. Sintaksisnya ditulis dengan menyebutkan nama identifier, diikuti operator =, ekspresi pengevaluasi baru, serta diakhiri tanda ;. Contoh: kondisi = false;.
- c. **Pernyataan Percabangan (if-else):** Mekanisme kontrol untuk mengarahkan jalurnya eksekusi kode berdasarkan hasil pengujian kondisi boolean. Struktur wajib menggunakan kata kunci if, diikuti oleh parameter pengujian di dalam kurung biasa (ekspresi), dan diikuti oleh blok perintah positif di dalam kurung kurawal {Statements}. Blok else {Statements} dapat ditambahkan di bawahnya secara opsional untuk menangani kondisi apabila hasil pengujian bernilai false.
- d. **Pernyataan print():** Berperan sebagai satu-satunya fungsi bawaan (*built-in*) yang bertugas menampilkan nilai variabel atau hasil akhir evaluasi ekspresi ke lingkungan

luar konsol. Pernyataan ini ditulis dengan kata kunci `print`, diikuti ekspresi di dalam kurung, dan ditutup dengan tanda `;`.

5. Mekanisme Komentar Kode

SimpleLog menyediakan fitur penulisan dokumentasi internal di dalam file kode sumber melalui mekanisme komentar satu baris. Pemrogram dapat menggunakan penanda berupa simbol minus ganda (`--`). Ketika pemindai leksikal mendeteksi tanda tersebut, seluruh karakter yang berada di belakangnya dalam satu baris yang sama akan langsung diabaikan dan tidak dimasukkan ke dalam daftar token. Komentar ini tidak akan memengaruhi pembentukan struktur *Abstract Syntax Tree* (AST) maupun hasil pembuatan kode antara.

1.4 Batasan Sistem

Dalam setiap perancangan dan pengembangan perangkat lunak, khususnya yang bersifat eksperimental dan berorientasi pada pembelajaran akademik, pendefinisian batasan ruang lingkup sangatlah krusial. Kompiler SimpleLog yang dibangun dalam proyek ini memiliki beberapa batasan yang disengaja untuk menjaga fokus pada konsep inti kompilasi. Pembatasan sistem ini diterapkan bukan sebagai sebuah kekurangan fungsional, melainkan sebagai sebuah keputusan arsitektural. Dengan membatasi kompleksitas fitur, analisis terhadap tahapan pemindaian leksikal, penguraian sintaksis, validasi semantik, hingga penerapan optimasi kode dapat dilakukan secara lebih komprehensif, mendalam, dan terarah.

Secara terperinci, batasan-batasan sistem yang ditetapkan dan diberlakukan pada proyek kompilator bahasa SimpleLog ini meliputi:

1. Eksklusivitas Sistem Tipe Data

Bahasa pemrograman SimpleLog beroperasi secara ketat dan eksklusif hanya menggunakan nilai logika. Oleh karena itu, sistem sama sekali tidak mendukung tipe data selain boolean. Kompilator tidak memiliki modul penanganan, pengenalan, maupun alokasi memori untuk tipe data numerik seperti *integer* dan *float*, serta tidak dapat memproses tipe data teks seperti *string*. Batasan ini secara signifikan menyederhanakan tahap analisis semantik, khususnya dalam proses pemeriksaan kesesuaian tipe data (*type checking*).

2. Absennya Struktur Kendali Iterasi

Berbeda dengan bahasa pemrograman *general-purpose* yang umum digunakan, arsitektur tata bahasa ini tidak mendukung perulangan (*loop*) seperti `while` atau `for`. Alur kontrol eksekusi program hanya direpresentasikan secara linear dan murni bergantung pada percabangan kondisional. Hal ini mengeliminasi kebutuhan kompilator untuk menangani algoritma lompatan balik (*backward jump*) dan evaluasi kondisi perulangan secara berkesinambungan.

3. Peniadaan Fitur Modularitas Subrutin

Desain sintaksis bahasa ini difokuskan pada eksekusi instruksi langsung pada *global scope*. Dengan demikian, sistem ini tidak mendukung deklarasi fungsi atau prosedur. Ketiadaan fitur ini membebaskan kompilator dari beban untuk mengelola manajemen lingkungan eksekusi kompleks, seperti penanganan *activation record*, pergerakan *call stack*, tingkat visibilitas variabel lokal yang bersarang, maupun mekanisme pertukaran argumen (*parameter passing*).

4. Batasan Titik Akhir Pipeline Kompilasi

Keseluruhan siklus pemrosesan kompilator SimpleLog ini diakhiri secara formal pada tahap *Intermediate Code Generator* (ICG) dan tahap Optimasi. Tahap *code generator* ke bahasa target (seperti Python/JavaScript) tidak diimplementasikan karena bersifat opsional/bonus pada ketentuan tugas. Sehingga, representasi keluaran atau *output* final dari sistem kompilator ini terbatas pada bentuk *Three Address Code* (TAC) yang telah disederhanakan, tanpa ada penerjemahan lebih lanjut ke *assembly language*, bahasa mesin, maupun skrip siap eksekusi.

BAB II

DESAIN

2.1 Arsitektur Sistem

Kompiler SimpleLog dibangun dengan mengadopsi arsitektur *pipeline* klasik, di mana seluruh tahapan pemrosesan terhadap kode sumber dilakukan secara berurutan melalui lima tahap utama. Dalam model arsitektur ini, aliran data bergerak secara linear dan sekuensial, di mana setiap tahapan dirancang untuk menerima komponen keluaran (*output*) dari tahapan sebelumnya untuk kemudian digunakan sebagai masukan (*input*) utama pada tahapan berjalan. Guna menjaga integritas hasil kompilasi, sistem ini dilengkapi dengan mekanisme terminasi dini. Apabila sebuah kesalahan (*error*) terdeteksi pada salah satu tahapan, maka seluruh proses kompilasi akan dihentikan lebih awal. Langkah penghentian ini diambil agar kompiler tidak membuang sumber daya untuk melanjutkan proses ke tahapan berikutnya dengan membawa representasi kode yang sudah tidak valid.

Secara visual, jalannya aliran data dan tahapan pemrosesan di dalam arsitektur *pipeline* kompiler SimpleLog dikonstruksikan melalui poin-poin berikut:

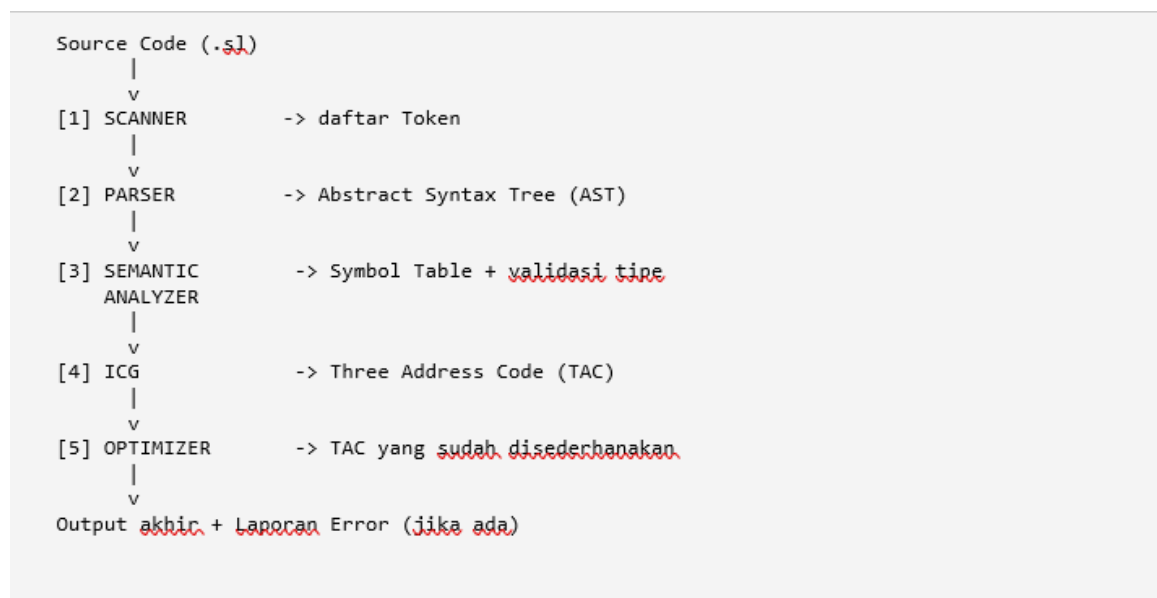
1. Source Code (.sl) → [1] SCANNER: File kode sumber mentah dengan ekstensi .sl menjadi masukan pertama yang masuk ke dalam modul *Scanner* untuk melalui analisis leksikal, yang bertugas memindai karakter demi karakter menjadi daftar Token yang sah.
2. Daftar Token → [2] PARSER: Rangkaian daftar Token yang telah dihasilkan oleh *Scanner* kemudian dialirkan menuju modul *Parser* untuk dianalisis kesesuaian strukturnya dengan aturan tata bahasa bebas konteks hingga membentuk pohon sintaks abstrak atau *Abstract Syntax Tree* (AST).
3. Abstract Syntax Tree (AST) → [3] SEMANTIC ANALYZER: AST yang telah terbentuk dikirimkan ke modul *Semantic Analyzer* untuk melalui proses validasi tipe data serta pengisian data ke dalam *Symbol Table* guna memastikan tidak ada aturan semantik logika bahasa yang dilanggar.
4. Symbol Table + Validasi Tipe → [4] ICG: Pohon sintaks yang telah dinyatakan valid secara semantik beserta data tabel simbol kemudian diproses oleh *Intermediate Code Generator* (ICG) untuk ditransformasikan ke dalam representasi kode antara yang disebut *Three Address Code* (TAC).
5. Three Address Code (TAC) → [5] OPTIMIZER: Instruksi TAC mentah hasil dari generator kode antara dialirkan ke dalam modul *Optimizer* untuk dieksekusi melalui berbagai teknik penyederhanaan kode guna menghasilkan TAC yang sudah disederhanakan dan jauh lebih efisien.

Melalui urutan *pipeline* di atas, sistem pada akhirnya akan mengeluarkan output akhir yang bersih beserta dengan laporan kesalahan apabila selama perjalanan eksekusi ditemukan kendala pada kode sumber.

Di samping kelima tahapan utama tersebut, aspek krusial yang menopang keandalan kompilator ini adalah modul *Error Handling*. Modul penanganan kesalahan ini sengaja tidak dirancang untuk berdiri sendiri sebagai sebuah tahapan terpisah di ujung aliran *pipeline*. Sebaliknya, modul ini dibangun secara melekat dan terintegrasi secara penuh di

dalam setiap tahapan pemrosesan yang ada. Dengan rancangan seperti ini, setiap tahapan memiliki kapabilitas untuk mendeteksi dan memicu jenis *error* spesifik yang sesuai dengan domain tugasnya, seperti *error* leksikal pada tahap *Scanner*, *error* sintaksis pada tahap *Parser*, serta *error* semantik pada tahap *Semantic Analyzer*.

Ketika kesalahan berhasil diidentifikasi oleh masing-masing modul, data kesalahan tersebut akan segera dikumpulkan dan dikelola oleh kelas khusus yang bernama *ErrorReporter*. Kelas *ErrorReporter* bertanggung jawab untuk menormalisasi seluruh temuan kesalahan tersebut ke dalam format pesan yang konsisten dan informatif bagi pemrogram. Laporan kesalahan yang ditampilkan ke layar konsol tidak hanya berupa teks pemberitahuan biasa, melainkan disajikan secara detail dengan menyertakan nomor baris yang tepat, letak posisi kolom terjadinya kesalahan, serta cuplikan atau potongan kode sumber yang bersangkutan. Keberadaan komponen ini memastikan pengguna dapat melakukan proses pelacakan kesalahan (*debugging*) secara cepat, efisien, dan akurat.



Gambar 1 Arsitektur pipeline kompiler SimpleLog

Tata Bahasa Grammar (BNF)

Tata bahasa bebas konteks (*Context-Free Grammar*) untuk bahasa pemrograman buatan SimpleLog didefinisikan secara formal menggunakan notasi BNF (*Backus-Naur Form*). Notasi formal ini berfungsi sebagai fondasi teoretis sekaligus cetak biru arsitektural yang menjadi acuan utama bagi tim pengembang dalam merancang dan mengimplementasikan seluruh algoritma pada modul *Parser*. Melalui spesifikasi tata bahasa ini, struktur penulisan kode dapat divalidasi secara matematis untuk memastikan aliran instruksi yang masuk memenuhi aturan sintaksis yang legal.

Berikut adalah spesifikasi tata bahasa bebas ko

nteks lengkap untuk bahasa SimpleLog yang direpresentasikan melalui notasi BNF:

```

<program>      ::= <statement>*

<statement>    ::= <declaration>
                  | <assignment>
                  | <if_stmt>
                  | <print_stmt>

<declaration>  ::= 'boolean' <identifier> '=' <bool_expr> ';'

<assignment>   ::= <identifier> '=' <bool_expr> ';'

<if_stmt>       ::= 'if' '(' <bool_expr> ')' '{' <statement>* '}'
                  [ 'else' '{' <statement>* '}' ]

<print_stmt>    ::= 'print' '(' <bool_expr> ')' ';'

<bool_expr>     ::= <bool_term> ( 'OR' <bool_term> )*

<bool_term>     ::= <bool_factor> ( 'AND' <bool_factor> )*

<bool_factor>   ::= 'NOT' <bool_factor>
                  | '(' <bool_expr> ')'
                  | <bool_literal>
                  | <identifier>

<bool_literal>  ::= 'true' | 'false'

<identifier>    ::= [a-zA-Z][a-zA-Z0-9]*

```

Gambar 2 Grammar BNF lengkap bahasa SimpleLog

Struktur tata bahasa di atas dirancang secara berjenjang menggunakan pendekatan arsitektur tata bahasa berlapis (*layered grammar structure*), yang terlihat jelas pada rantai penurunan ekspresi logika, yaitu dari <bool_expr> menuju <bool_term>, dan bermuara pada <bool_factor>. Desain pelapisan ini sengaja dipilih agar tingkatan kekuatan pengikatan operator logika (*operator precedence*) dapat tertanam secara implisit dan langsung di dalam hierarki sintaksis bahasa itu sendiri. Dengan metode ini, kompilator tidak lagi membutuhkan tabel prioritas eksternal atau logika tambahan di luar struktur pohon sintaks untuk mengurai ekspresi biner bersarang.

Melalui mekanisme penurunan berlapis tersebut, urutan eksekusi yang dihasilkan secara otomatis mematuhi konvensi standar logika matematika universal, dengan urutan tingkat kekuatan: NOT (prioritas tertinggi) → AND (prioritas menengah) → OR (prioritas terendah). Struktur ini sangat krusial bagi implementasi *Recursive Descent Parser*, karena metode penguraian *top-down* akan mengevaluasi aturan dengan prioritas terendah (OR) di tingkat terluar, sedangkan aturan dengan prioritas tertinggi (NOT dan nilai literal) akan ditarik ke tingkat terdalam dari *Abstract Syntax Tree* (AST), sehingga dieksekusi paling awal saat proses evaluasi nilai dijalankan.

2.2 Desain Token

Tahap pertama dalam seluruh siklus *pipeline* kompilasi adalah proses analisis leksikal yang dieksekusi oleh modul *Scanner*. Pada tahapan ini, teks kode sumber yang masih berupa deretan karakter mentah yang tidak terstruktur akan dipindai, dipecah, dan dikelompokkan menjadi unit-unit leksikal terkecil yang memiliki makna komputasional. Unit-unit bermakna inilah yang disebut sebagai token. Desain arsitektur token yang presisi

sangat menentukan keberhasilan dan kemudahan kerja modul *Parser* pada tahap selanjutnya. Dengan adanya tokenisasi, *Parser* tidak perlu lagi melakukan pemrosesan tingkat rendah terhadap spasi, tabulasi, atau pemecahan kata, melainkan dapat langsung mengevaluasi aliran data melalui urutan token yang terstruktur.

Dalam implementasi bahasa SimpleLog, seluruh karakter dan kata yang diakui secara legal oleh *Scanner* diklasifikasikan secara ketat ke dalam beberapa kategori token spesifik. Pengklasifikasian ini mencakup identifikasi kata kunci bawaan (*keywords*), simbol operasional, simbol struktural, pengenalan dinamis (*identifier*), hingga instrumen penanda akhir dokumen.

Berikut adalah spesifikasi desain token yang didukung oleh kompilator SimpleLog, beserta contoh nilai representatif dari masing-masing kategori leksikal:

Tabel 1 Daftar jenis token bahasa SimpleLog

Kategori	Token	Contoh
Keyword tipe	BOOLEAN	boolean
Keyword literal	TRUE, FALSE	true, false
Keyword operator	AND, OR, NOT	AND, OR, NOT
Keyword kendali	IF, ELSE	if, else
Keyword fungsi	PRINT	print
Simbol	ASSIGN, SEMICOLON	=, ;
Simbol	LPAREN, RPAREN	(,)
Simbol	LBACE, RBACE	{, }
Identifier	IDENTIFIER	a, hasil, flag1
Komentar	COMMENT	-- ini komentar
Penanda akhir	EOF	(akhir berkas)

2.3 Desain AST (Abstract Syntax Tree)

Setelah melalui tahap penguraian (*parsing*), rangkaian token yang valid akan ditransformasikan menjadi struktur pohon hierarkis yang disebut *Abstract Syntax Tree* (AST). AST ini merepresentasikan struktur sintaksis program secara abstrak, yang berarti elemen-elemen leksikal yang tidak memengaruhi semantik program (seperti tanda titik koma atau kurung pembatas) akan dieliminasi. Di dalam kompilator SimpleLog, setiap konstruksi bahasa diwakili oleh jenis *node* (simpul) AST tersendiri yang memiliki peran spesifik.

Berikut adalah rancangan struktur *node* AST yang digunakan untuk memetakan sintaksis bahasa SimpleLog:

Tabel 2 Struktur node AST SimpleLog

Node	Merepresentasikan
ProgramNode	Akar pohon, berisi seluruh daftar pernyataan
DeclarationNode	Deklarasi variabel boolean baru
AssignmentNode	Pemberian nilai baru ke variabel yang sudah ada
IfNode	Struktur kendali if-else beserta blok then dan else
PrintNode	Pernyataan print untuk menampilkan nilai
BinaryOpNode	Operasi biner AND / OR antara dua operand
UnaryOpNode	Operasi unary NOT pada satu operand
LiteralNode	Nilai literal true / false
IdentifierNode	Referensi ke nama variabel

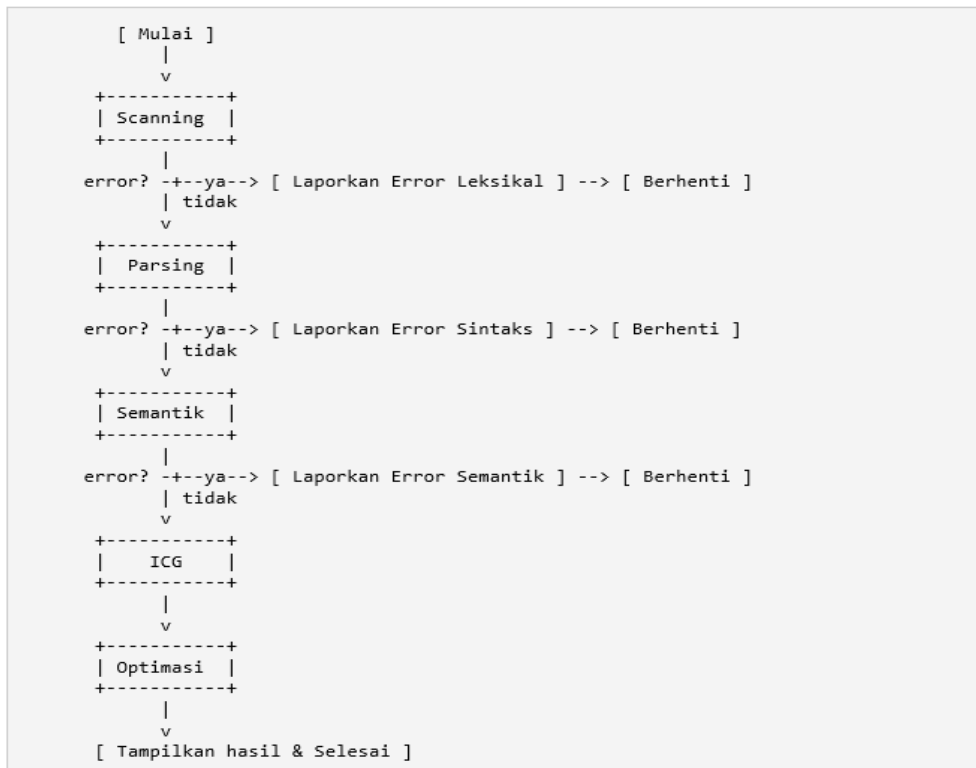
Melalui desain AST di atas, struktur program dapat dijelaskan secara logis dan terarah. ProgramNode bertindak sebagai pondasi utama (akar) yang mengelompokkan semua jenis pernyataan seperti deklarasi, penugasan, maupun ekspresi cetak.

Ketika pemrogram menuliskan operasi logika yang kompleks, *Parser* akan memecahnya secara hierarkis menjadi simpul operasi biner (BinaryOpNode) untuk operator AND/OR, atau simpul operasi unary (UnaryOpNode) khusus untuk operator negasi NOT. Operasi-operasi tersebut pada akhirnya akan mengevaluasi simpul daun (*leaves*) yang berupa variabel (IdentifierNode) atau nilai logika pasti (LiteralNode). Desain yang terstruktur dan bersih ini sangat krusial karena mempermudah modul *Semantic Analyzer* serta *Intermediate Code Generator* (ICG) untuk melakukan penelusuran pohon (*tree traversal*) secara valid pada tahapan berikutnya.

2.4 Diagram Alir Proses Kompilasi

Diagram alir proses kompilasi menggambarkan seluruh urutan logika, mekanisme pengambilan keputusan, serta aliran kendali yang terjadi di dalam kompilator SimpleLog dari awal hingga selesai. Melalui diagram alir ini, dapat diidentifikasi secara jelas titik-titik krusial di mana proses kompilasi akan diteruskan, serta kondisi-kondisi bersyarat yang memaksa sistem untuk melakukan terminasi dini apabila ditemukan galat pada kode sumber. Proses kompilasi diinisialisasi ketika kode sumber dimasukkan untuk diproses oleh modul Scanning. Pada tahap awal ini, sistem akan melakukan pengujian untuk memeriksa apakah terdapat error leksikal. Jika ditemukan kesalahan, aliran kendali akan langsung diarahkan untuk melaporkan error leksikal tersebut dan sistem akan segera berhenti beroperasi demi mencegah pemrosesan kode yang cacat. Namun, jika tidak ada kesalahan leksikal yang terdeteksi, aliran data akan diteruskan ke tahap Parsing. Di dalam fase penguraian ini, sistem kembali melakukan pemeriksaan terhadap potensi munculnya error sintaksis. Apabila terdeteksi kesalahan struktur tata bahasa, sistem akan melaporkan error sintaks dan langsung mengakhiri seluruh rangkaian kompilasi secara paksa.

Sebaliknya, jika struktur kode dinyatakan valid, hasil penguraian tersebut akan dikirimkan ke modul Semantik. Pada fase analisis semantik ini, sistem akan menguji kembali kelayakan aturan bahasa program, di mana sistem akan melaporkan error semantik serta menghentikan proses jika ditemukan pelanggaran aturan seperti variabel ganda atau tidak terdefinisi. Setelah kode sumber berhasil melewati ketiga gerbang validasi kesalahan tersebut tanpa hambatan, aliran program akan memasuki tahap ICG untuk menghasilkan representasi kode antara. Kode antara mentah tersebut kemudian langsung dialirkan ke dalam modul Optimasi untuk disederhanakan instruksinya guna meningkatkan efisiensi. Siklus kompilasi ini akhirnya mencapai puncaknya ketika kompilator menampilkan hasil akhir yang telah bersih dari galat dan dioptimasi, menandakan bahwa seluruh proses kompilasi telah diselesaikan dengan sukses.



Gambar 3 Diagram alir proses kompilasi SimpleLog

BAB III

IMPLEMENTASI

3.1 Modul Scanner (scanner.py)

Modul Scanner bertugas mengubah teks sumber mentah menjadi rangkaian token yang bermakna. Implementasi modul ini menggunakan pendekatan pembacaan karakter demi karakter (*character-by-character scanning*) yang dikombinasikan dengan pelacakan posisi baris dan kolom secara presisi. Hal ini dilakukan untuk mendukung kemunculan pesan *error* leksikal yang akurat apabila pengguna melakukan kesalahan pengetikan karakter.

Secara arsitektural, terdapat empat komponen utama yang menyusun modul pemindai ini:

1. **Enum TokenType:** Digunakan untuk mendefinisikan dan mendaftarkan seluruh jenis kategori token yang valid di dalam bahasa SimpleLog.
2. **Dataclass Token:** Berperan sebagai struktur data untuk menyimpan informasi setiap token yang berhasil dipindai, meliputi tipe token, nilai teks aslinya, beserta posisi baris dan kolomnya.
3. **Kelas Scanner:** Merupakan inti operasional modul yang menggunakan metode `tokenize()` sebagai *entry point*. Di dalam kelas ini terdapat metode `_scan_token()` yang bertindak sebagai *dispatcher* utama untuk menyeleksi dan menangani karakter *whitespace* (spasi kosong), penanda komentar, simbol operasional, hingga membedakan *identifier* dengan *keyword* bawaan sistem.
4. **Kelas LexicalError:** Komponen khusus yang bertugas merepresentasikan objek kesalahan leksikal yang dipicu saat *Scanner* menemukan karakter yang tidak dikenal atau ilegal.

Berikut adalah potongan kode kunci dari implementasi *dispatcher* utama yang mengeksekusi logika pemilahan karakter di atas:

```
def _scan_token(self):
    ch = self._current()
    if ch in '\t\r\n':
        self._advance(); return
    if ch == '-' and self._peek() == '-':
        self._scan_comment(line, col); return
    if ch in SYMBOLS:
        self._advance()
        self._add_token(SYMBOLS[ch], ch, line, col); return
    if ch.isalpha() or ch == '_':
        self._scan_identifier(line, col); return
    # karakter tidak dikenal -> error leksikal
    self._advance()
    self.errors.append(LexicalError(...))
```

Gambar 4 Dispatcher utama pada kelas Scanner

Implementasi kode scanner.py

1. Definisi Kamus Token (Keywords & Symbols)

Tabel 3 Spesifikasi Token untuk Keyword dan Simbol pada Kompilator SimpleLog

```
# Pemetaan keyword spesifik SimpleLog → TokenType
KEYWORDS = {
    'boolean' : TokenType.BOOLEAN,
    'true'     : TokenType.TRUE,
    'false'    : TokenType.FALSE,
    'AND'      : TokenType.AND,
    'OR'       : TokenType.OR,
    'NOT'      : TokenType.NOT,
    'if'       : TokenType.IF,
    'else'     : TokenType.ELSE,
    'print'    : TokenType.PRINT,
}

# Pemetaan simbol struktural → TokenType
SYMBOLS = {
    '=' : TokenType.ASSIGN,
    ';' : TokenType.SEMICOLON,
    '(' : TokenType.LPAREN,
    ')' : TokenType.RPAREN,
    '{' : TokenType.LBRACE,
    '}' : TokenType.RBRACE,
}
```

2. Fungsi Pemilah Utama (Dispatcher)

Tabel 4 Kode Sumber Penanganan Karakter dan Pemicu Error Leksikal

```
def _scan_token(self):
    """Baca satu token dari posisi sekarang dan
    tentukan jenisnya."""
    line = self.line
    col  = self.column
    ch   = self._current()

    # 1. Lewati karakter whitespace (spasi, enter,
    tab)
    if ch in ' \t\r\n':
        self._advance()
        return

    # 2. Tangani komentar (mengabaikan teks setelah
    tanda --)
    if ch == '-' and self._peek() == '-':
        self._scan_comment(line, col)
        return

    # 3. Pindai simbol struktural / operator
    if ch in SYMBOLS:
        self._advance()
        self._add_token(SYMBOLS[ch], ch, line, col)
        return
```

```

        # 4. Pindai identifier atau keyword
        if ch.isalpha() or ch == '_':
            self._scan_identifier(line, col)
            return

        # 5. Pemicu Error Leksikal untuk karakter di
        luar alfabet/symbol valid
        self._advance()
        err = LexicalError(f"Karakter tidak dikenal:
        '{ch}'", line, col)
        self.errors.append(err)

```

3.2 Modul Parser (parser.py)

Modul Parser bertugas mengimplementasikan teknik *recursive descent parsing*, di mana setiap aturan tata bahasa (grammar BNF) yang telah didefinisikan sebelumnya dipetakan secara langsung menjadi sebuah metode Python tersendiri. Pendekatan arsitektural ini sangat menguntungkan karena membuat struktur kode Parser menjadi sangat representatif dan mudah ditelusuri kembali ke bentuk grammar aslinya pada Bab 2.

Dalam memproses ekspresi logika dan menjaga hierarki prioritas operator, modul ini mendefinisikan tiga metode pemrosesan utama yang berjalan secara berjenjang:

1. **_parse_bool_expr():** Bertugas mengevaluasi dan menangani operator disjungsi OR, yang menempati hierarki prioritas terendah dalam tata bahasa.
2. **_parse_bool_term():** Bertugas mengevaluasi operator konjungsi AND, yang memiliki hierarki prioritas menengah.
3. **_parse_bool_factor():** Bertugas mengevaluasi ekspresi dengan prioritas tertinggi, yang mencakup operator negasi NOT, pengelompokan menggunakan tanda kurung, pembacaan nilai literal, serta identifikasi variabel.

Keunggulan lain dari Parser ini adalah tersedianya mekanisme pemulihan kesalahan atau *panic-mode recovery* melalui implementasi fungsi `_sync()`. Fitur pemulihan ini secara cerdas memungkinkan Parser untuk tetap melanjutkan proses analisis dan melaporkan lebih dari satu *error* sintaksis dalam satu kali eksekusi, alih-alih langsung berhenti beroperasi pada *error* pertama yang ditemukan.

Berikut adalah potongan kode kunci yang mendemonstrasikan implementasi pembentukan *Abstract Syntax Tree* (AST) sekaligus penjagaan hierarki prioritas operator melalui struktur pemanggilan rekursif:

```
def _parse_bool_expr(self):      # OR - prioritas terendah
    left = self._parse_bool_term()
    while self._check(TokenType.OR):
        op_tok = self._advance()
        right = self._parse_bool_term()
        left = BinaryOpNode(op='OR', left=left, right=right, ...)
    return left

def _parse_bool_factor(self):    # NOT - prioritas tertinggi
    if tok.type == TokenType.NOT:
        self._advance()
        operand = self._parse_bool_factor() # rekursif
        return UnaryOpNode(op='NOT', operand=operand, ...)
    ...
```

Gambar 5 Implementasi prioritas operator melalui struktur rekursif

Implementasi kode parser.py

1. Struktur Node AST

Tabel 5 Kode Sumber Struktur Data Pohon Sintaksis Abstrak (Abstract Syntax Tree) Bahasa SimpleLog

```
# — Statement nodes
_____

@dataclass
class IfNode (ASTNode):
    """
    Representasi percabangan logika.
    Struktur: if (<cond>) { ... } [else { ... }]
    """
    condition: ASTNode = None
    then_branch: list =
    field(default_factory=list)
    else_branch: Optional[list] = None

@dataclass
class BinaryOpNode (ASTNode):
    """
    Representasi operasi logika (AND/OR).
    Struktur: <left> op <right>
    """
    op: str = ''
    left: ASTNode = None
    right: ASTNode = None
```

2. Mekanisme Pemulihan Error (*Panic-Mode Recovery*)

Tabel 6 Kode Sumber Mekanisme Pemulihan Sintaks (Syntax Error Recovery) pada Parser SimpleLog

```
def _sync(self):
    """
    Panic-mode recovery: lewati token sampai ketemu
    titik awal statement baru agar parsing bisa lanjut.
```

```

        """
        while not self._check(TokenType.EOF):
            # Mencari token yang biasanya mengawali sebuah
            statement baru
            if self._current().type in (
                TokenType.BOOLEAN, TokenType.IF,
                TokenType.PRINT, TokenType.RBRACE
            ):
                return # Sinkronisasi berhasil, lanjutkan
                proses parsing

            # Buang token yang menyebabkan kebingungan
            self.advance()

```

3.3 Modul Semantic Analyzer (semantic.py)

Modul Semantic Analyzer memiliki peran krusial dalam memvalidasi kebenaran logika program setelah pohon sintaks berhasil dibentuk. Modul ini diimplementasikan menggunakan pola desain *visitor*, di mana setiap tipe *node* pada AST memiliki metode penanganannya tersendiri, seperti `_visit_NamaNode()`. Terdapat dua tanggung jawab utama dari fase analisis ini, yaitu membangun *Symbol Table* dan memvalidasi setiap aturan penggunaan variabel di dalam kode sumber.

Secara rinci, komponen dan tugas operasional dalam modul ini meliputi:

1. **Pengelolaan *Symbol Table*:** Berfungsi sebagai pusat penyimpanan informasi bagi setiap variabel yang terdaftar. Data yang disimpan mencakup nama variabel, tipe data yang mengikatnya, letak baris deklarasi, hingga status inisialisasi nilainya.
2. **Validasi Semantik Aturan:** Modul ini menerapkan pemeriksaan ketat untuk menjamin keamanan jalannya program. Validasi yang diterapkan meliputi pencegahan terhadap variabel yang belum dideklarasikan, deteksi deklarasi ganda pada variabel yang sama, pelarangan *assignment* tanpa deklarasi sebelumnya, serta penolakan terhadap penggunaan variabel yang dipanggil sebelum diinisialisasi.

Berikut adalah potongan kode kunci yang mendemonstrasikan mekanisme validasi pada saat *Semantic Analyzer* mengevaluasi penggunaan sebuah *identifier*:

```

def _visit_IdentifierNode(self, node):
    sym = self.symbol_table.lookup(node.name)
    if sym is None:
        self._report(f"Variabel '{node.name}' belum "
                    f"dideklarasikan", node.line)
        return 'boolean'
    if not sym.initialized:
        self._report(f"Variabel '{node.name}' digunakan "
                    f"sebelum diinisialisasi", node.line)
    return sym.type_

```

Gambar 6 Validasi identifier pada Semantic Analyzer

Implementasi kode semantic.py

1. Pengelolaan Symbol Table

Tabel 7 Kode Sumber Validasi Deklarasi Ganda pada Tahap Analisis Semantik

```
class SymbolTable:
    def __init__(self):
        self._table: dict[str, Symbol] = {}

    def declare(self, name: str, type_: str, line: int)
    -> Optional[str]:
        """
        Daftarkan variabel baru.
        Kembalikan pesan error jika sudah
        dideklarasikan sebelumnya.
        """
        # Validasi: Tolak jika nama variabel sudah ada
        (Deklarasi Ganda)
        if name in self._table:
            prev = self._table[name]
            return (f"Variabel '{name}' sudah
            dideklarasikan "
                    f"di baris {prev.declared_at}")

        # Daftarkan variabel baru ke dalam tabel simbol
        self._table[name] = Symbol(name=name,
            type_=type_, declared_at=line)
        return None
```

2. Validasi Penggunaan Variabel

Tabel 8 Kode Sumber Pemeriksaan Aturan Semantik Penggunaan Variabel Bahasa SimpleLog

```
def _visit_IdentifierNode(self, node:
IdentifierNode) -> str:
    sym = self.symbol_table.lookup(node.name)

    # Validasi 1: Variabel belum dideklarasikan
    sama sekali
    if sym is None:
        self._report(
            f"Variabel '{node.name}' belum
            dideklarasikan",
            node.line
        )
        return 'boolean' # asumsikan boolean agar
        analisis lanjut

    # Validasi 2: Dideklarasikan tapi belum
    diinisialisasi
    if not sym.initialized:
        self._report(
            f"Variabel '{node.name}' digunakan sebelum
            diinisialisasi",
            node.line
        )
```

```
return sym.type
```

3.4 Modul ICG (Intermediate Code Generator (icg.py))

Modul ICG memiliki peran penting untuk menghasilkan representasi *Three Address Code* (TAC) dari struktur *Abstract Syntax Tree* (AST) yang telah divalidasi. Pada tahap ini, setiap ekspresi logika yang kompleks dipecah dan disederhanakan menjadi rangkaian instruksi dasar dengan memanfaatkan variabel *temporary* (seperti t0, t1, dan seterusnya) serta label penanda lompatan (seperti L0, L1, dan seterusnya).

Salah satu fitur teknis paling esensial yang diimplementasikan pada modul ini adalah mekanisme *short-circuit evaluation* untuk menangani eksekusi operator AND dan OR. Modul ini tidak memproses operator AND/OR tersebut secara langsung pada level TAC, melainkan menerjemahkannya menggunakan instruksi lompatan bersyarat, seperti `iffalse ... goto` dan `if ... goto`. Pendekatan arsitektural ini diterapkan karena secara akurat meniru perilaku dan cara kompilator nyata di industri dalam mengevaluasi logika boolean.

Berikut adalah contoh hasil representasi TAC yang dibangkitkan oleh modul ICG dari program pada Kode 1.1 sebelum melewati tahap optimasi:

```
0:  t0 = true
1:  a = t0
2:  t1 = false
3:  b = t1
4:  iffalse a goto L0
5:  t3 = NOT b
6:  iffalse t3 goto L0
7:  t2 = true
8:  goto L1
9:  L0:
10: t2 = false
11: L1:
12: iffalse t2 goto L2
13: print a
14: goto L3
15: L2:
16: print b
17: L3:
```

Gambar 7 Three Address Code hasil ICG (18 instruksi, sebelum optimasi)

Implementasi kode `icg.py`

1. Struktur TAC

Tabel 9 Kode Sumber Pemformatan String untuk Instruksi Tiga Alamat (Three Address Code)

```
class TACInstruction:
    # operator atau instruksi dasar
    op      : str
    # operand pertama
    arg1     : str = ''
    # operand kedua (opsional)
    arg2     : str = ''
    # tempat menyimpan hasil (opsional)
```

```

result : str = ''

def __str__(self) -> str:
    if self.op == 'LABEL':
        return f"{self.result}:"
    if self.op == 'GOTO':
        return f"    goto {self.result}"
    if self.op == 'IF_GOTO':
        return f"    if {self.arg1} goto
{self.result}"
    if self.op == 'COPY':
        return f"    {self.result} = {self.arg1}"
# ... lanjutan representasi string lainnya ...

```

2. Evaluasi Logika Bersyarat (Short-Circuit)

Tabel 10 Kode Sumber Generasi TAC untuk Ekspresi Logika Optimasi Melompat

```

# Jika keduanya lolos, hasilkan true
self._emit('COPY', arg1='true',
result=result)
self._emit('GOTO', result=l_end)

self._emit('LABEL', result=l_false)
self._emit('COPY', arg1='false',
result=result)

self._emit('LABEL', result=l_end)

```

3. Penanganan Blok If-Else

Tabel 11 Kode Sumber Pemicu Generator Kode Alur Blok Kontrol If-Else

```

def _visit_IfNode(self, node: IfNode) -> str:
    cond = self._visit(node.condition)
    l_else = self._new_label()
    l_end = self._new_label()

    # Lompat ke blok else jika kondisi FALSE
    self._emit('IFFALSE_GOTO', arg1=cond,
result=l_else)

    # Generate TAC untuk blok 'then'
    for stmt in node.then_branch:
        self._visit(stmt)

    # Lompat ke end (melewati blok else)
    self._emit('GOTO', result=l_end)

    # Generate TAC untuk blok 'else' (jika ada)
    self._emit('LABEL', result=l_else)
    if node.else_branch:
        for stmt in node.else_branch:
            self._visit(stmt)

```

```
self._emit('LABEL', result=l_end)
return ''
```

3.5 Modul Optimasi (optimizer.py)

Modul Optimasi bertujuan untuk menyederhanakan dan meningkatkan efisiensi dari representasi *Three Address Code* (TAC) yang telah dihasilkan oleh tahap sebelumnya. Pada modul ini, dua teknik optimasi utama diterapkan secara iteratif atau diulang secara terus-menerus hingga tidak ditemukan lagi perubahan pada kumpulan instruksi. Pendekatan iteratif ini sangat penting untuk dilakukan karena hasil penyederhanaan dari satu teknik seringkali mengekspos peluang optimasi baru bagi teknik yang lainnya.

Secara rinci, kedua teknik optimasi yang diimplementasikan pada modul ini adalah:

1. **Constant Folding:** Teknik ini bertugas mengevaluasi ekspresi logika yang seluruh nilai operandnya sudah diketahui secara pasti sejak awal pada saat proses kompilasi berlangsung. Terdapat sebuah mekanisme pengamanan yang krusial dalam implementasinya: nilai konstanta yang telah diketahui wajib di-reset ulang setiap kali sistem bertemu dengan instruksi LABEL. Hal ini diterapkan karena label merupakan titik temu dari berbagai kemungkinan jalur eksekusi percabangan yang berpotensi membawa nilai variabel yang berbeda. Tanpa adanya mekanisme pengamanan ini, proses *folding* sangat berisiko menghasilkan instruksi kode yang salah secara logika.
2. **Dead Code Elimination:** Teknik ini bertujuan untuk membersihkan TAC dengan cara menghapus instruksi *NOP* (*No Operation*) yang tersisa dari proses *folding* sebelumnya. Selain itu, algoritma ini juga bertugas mengeliminasi blok instruksi yang tidak dapat dicapai lagi (*unreachable code*) setelah adanya instruksi *goto* tanpa syarat, serta menghapus variabel *temporary* yang telah dihitung sistem namun tidak pernah dibaca atau digunakan kembali.

Sebagai bentuk pembuktian dari efektivitas kinerja modul ini, berikut adalah hasil optimasi dari instruksi TAC yang sebelumnya berjumlah 18 baris (pada Kode 3.4), di mana instruksi berhasil direduksi menjadi hanya 13 baris perintah operasional:

```
0:  a = true
1:  b = false
2:  t2 = true
3:  goto L1
4:  L0:
5:  t2 = false
6:  L1:
7:  ifFalse t2 goto L2
8:  print a
9:  goto L3
10: L2:
11: print b
12: L3:
```

Gambar 8 TAC setelah optimasi (13 instruksi, penghematan 28%)

Implementasi kode optimizer.py

1. Keamanan Constant Folding pada Label

Tabel 12 Kode Sumber Penjelajah Optimasi Konstanta pada Kompilator

```
def optimize(self, instructions:
list[TACInstruction]) -> list[TACInstruction]:
    known    : dict[str, str]      = {}
    result   : list[TACInstruction] = []
    folded   : int                 = 0

    for instr in instructions:
        # PENTING: Reset tabel nilai jika bertemu
LABEL      # karena label adalah titik temu berbagai
jalur percabangan
        if instr.op == 'LABEL':
            known.clear()
            result.append(instr)
            continue

        new_instr = self._fold(instr, known)
        result.append(new_instr)

    # ... lanjutan pembaruan tabel known ...
```

2. Logika Eliminasi Dead Code (Variabel Tidak Terpakai)

Tabel 13 Kode Sumber Pembersihan Variabel Temporer Tidak Terpakai

```
def _remove_unused_temps(self, instructions:
list[TACInstruction]) -> list[TACInstruction]:
    # 1. Hitung berapa kali tiap variabel digunakan
    sebagai argumen
    use_count: dict[str, int] = {}
    for instr in instructions:
        for arg in (instr.arg1, instr.arg2):
            if arg:
                use_count[arg] = use_count.get(arg,
0) + 1

    # 2. Hapus instruksi temporary (tN) jika
    use_count-nya 0
    result = []
    for instr in instructions:
        r = instr.result
        is_temp    = r.startswith('t') and
r[1:].isdigit()
        is_write   = instr.op in ('COPY', 'NOT',
'AND', 'OR')
        is_unused  = use_count.get(r, 0) == 0
```

```

        if is_temp and is_write and is_unused:
            continue # Buang instruksi (Dead
Code)
        result.append(instr)

    return result

```

3. Orkestrasi Iteratif (Loop)

Tabel 14 Kode Sumber Pengendali Loop Optimasi Tahap Backend Kompilator

```

class Optimizer:
    # ... inisialisasi ...

    def optimize(self, instructions:
list[TACInstruction]) -> list[TACInstruction]:
        current = instructions

        while True:
            self.passes += 1
            prev_len = len(current)

            # Pass 1: Constant Folding
            after_fold = self.folder.optimize(current)

            # Pass 2: Dead Code Elimination
            after_dce =
self.eliminator.optimize(after_fold)

            # Berhenti berulang jika panjang instruksi
sudah tidak menyusut
            if len(after_dce) == prev_len:
                current = after_dce
                break

            current = after_dce

        return current

```

3.6 Modul Error Handling & Integrasi (compiler.py)

Modul ini berfungsi sebagai titik pusat integrasi yang menyatukan seluruh aliran *pipeline* kompilasi. Di dalam modul ini, terdapat kelas `SimpleLogCompiler` yang memegang kendali penuh untuk mengeksekusi kelima tahapan utama secara berurutan, mulai dari analisis leksikal hingga proses optimasi akhir. Dengan adanya integrasi ini, aliran data antarmodul dapat berjalan secara mulus dan terstruktur.

Selain mengatur eksekusi tahapan, modul ini juga mengelola sistem pelaporan kesalahan secara terpusat melalui kelas `ErrorReporter`. Kelas tersebut bertugas menormalisasi berbagai *error* yang ditangkap dari setiap fase kompilasi menjadi sebuah format pesan yang konsisten dan rapi. Laporan kesalahan yang ditampilkan tidak hanya sekadar peringatan, melainkan dilengkapi dengan informasi komprehensif yang mencakup kategori kesalahan (Leksikal, Sintaks, atau Semantik), detail nomor baris dan posisi kolom

terjadinya galat, serta potongan kode sumber yang relevan untuk memudahkan proses perbaikan oleh pengguna.

Untuk mendukung fleksibilitas operasional, modul ini turut menyediakan Antarmuka Baris Perintah (*Command Line Interface* atau CLI) dengan berbagai opsi argumen tampilan. Fitur ini memungkinkan pengguna untuk melihat hasil evaluasi kompilator pada tahapan tertentu secara transparan dan detail.

Berikut adalah opsi penggunaan CLI yang didukung oleh kompilator SimpleLog:

```
python compiler.py program.sl          # kompilasi biasa
python compiler.py program.sl --tokens # + tampilkan token
python compiler.py program.sl --ast    # + tampilkan AST
python compiler.py program.sl --tac    # + tampilkan TAC
python compiler.py program.sl --optimize # + jalankan optimasi
python compiler.py program.sl --all    # tampilkan semuanya
python compiler.py --demo              # jalankan 4 skenario uji
```

Gambar 9 Opsi penggunaan CLI compiler.py

Implementasi kode compiler.py

1. Integrasi Pipeline Kompilasi

Tabel 15 Kode Sumber Pengendali Sekuensial Antarfase Kompilator Bahasa SimpleLog

```
class SimpleLogCompiler:
    # ... [Inisialisasi atribut lainnya] ...

    def compile(self, optimize: bool = True) -> bool:
        """
        Jalankan seluruh pipeline kompilasi secara
        sekuensial.
        Berhenti seketika jika ada fase yang gagal
        (menghasilkan False).
        """
        return (
            self._phase_scan()
            and self._phase_parse()
            and self._phase_semantic()
            and self._phase_icg()
            and (not optimize or
                self._phase_optimize())
        )
```

2. Sistem Pelaporan Error Terpusat

Tabel 16 Kode Sumber Penanganan dan Pemetaan Posisi Visual Eksplisit Kompilator

```
class ErrorReporter:
    def __init__(self, source: str):
        self.source = source.splitlines()
        self.errors : list[CompilerError] = []

    def add(self, phase: ErrorPhase, message: str,
            line: int, col: int = 0,
            severity: Severity = Severity.ERROR):
        """Normalisasi semua jenis galat menjadi format
        CompilerError standar."""
        self.errors.append(CompilerError(
            phase=phase, severity=severity,
            message=message, line=line, col=col
        ))

    def _print_source_context(self, line: int, col:
int):
        """Tampilkan baris kode sumber dan lokasi
        persis kursor error terjadi."""
        if line < 1 or line > len(self.source):
            return
        src_line = self.source[line - 1]
        print(f"          {line:>4} | {src_line}")
        if col > 0:
            pointer = " " * (col - 1) + "^"
            print(f"          | {pointer}")
```


BAB IV

PENGUJIAN

4.1 Test Case 1- Program Valid Dasar

Pengujian pertama ini bertujuan untuk memverifikasi fungsionalitas dan keandalan keseluruhan dari *pipeline* kompilator, mulai dari tahap *Scanner*, *Parser*, *Semantic Analyzer*, *ICG*, hingga *Optimizer*. Skenario uji ini dirancang menggunakan representasi kode sumber program SimpleLog dasar yang sepenuhnya valid dan sesuai dengan contoh pada spesifikasi tugas. Hal ini dilakukan untuk memastikan bahwa aliran data berjalan secara normal dan menghasilkan keluaran yang dieksekusi tanpa adanya *error* sama sekali.

Berikut adalah potongan kode sumber yang digunakan dalam pengujian validasi dasar ini:

```
boolean a = true;
boolean b = false;
if (a AND NOT b) {
    print(a);
} else {
    print(b);
}
```

Gambar 10 Hasil pengujian Test Case 1

Tabel 17 Hasil Pengujian Validasi Fungsional Kompilator Tanpa Error

Tahap	Hasil
Scanning	Berhasil — 33 token dihasilkan
Parsing	Berhasil — AST terbentuk tanpa error
Semantik	Berhasil — 2 variabel terdaftar (a, b)
ICG	Berhasil — 18 instruksi TAC
Optimasi	Berhasil — disederhanakan menjadi 13 instruksi
Output print	a (karena a AND NOT b bernilai true)

Tabel 18 Riwayat kompilasi lengkap dan penelusuran alur kode antar (TAC) Test Case 1

<pre>===== TEST CASE 1 – Program Valid Dasar ===== Source Code:</pre>
--

```

boolean a = true;
boolean b = false;
if (a AND NOT b) {
    print(a);
} else {
    print(b);
}

```

```

-----
[*] Scanning : Berhasil (33 token dihasilkan)
[*] Parsing  : Berhasil (AST terbentuk)
[*] Semantik : Berhasil (Variabel tervalidasi)
[*] ICG      : Berhasil (18 instruksi TAC mentah)

```

--- TAC Sebelum Optimasi ---

Three Address Code (TAC)

```

0:      t0 = true
1:      a = t0
2:      t1 = false
3:      b = t1
4:      ifFalse a goto L0
5:      t3 = NOT b
6:      ifFalse t3 goto L0
7:      t2 = true
8:      goto L1
9:  L0:
10:     t2 = false
11:  L1:
12:     ifFalse t2 goto L2
13:     print a
14:     goto L3
15:  L2:
16:     print b
17:  L3:

```

Total instruksi: 18

```

[*] Optimasi : Berhasil (13 instruksi setelah optimasi)

```

--- TAC Sesudah Optimasi ---

Three Address Code (TAC)

```

0:      a = true
1:      b = false
2:      t2 = true
3:      goto L1
4:  L0:
5:      t2 = false
6:  L1:
7:      ifFalse t2 goto L2

```

8:	print a
9:	goto L3
10:	L2:
11:	print b
12:	L3:
Total instruksi: 13	
Laporan Optimasi:	
Pass dijalankan	: 2
Instruksi di-fold	: 5
Instruksi dihapus (DCE)	: 5
Penghematan	: 5 instruksi (28%)

4.2 Test Case 2 – Error Leksikal

Pengujian ini bertujuan untuk memverifikasi kemampuan modul *Scanner* dalam mendeteksi keberadaan karakter yang tidak dikenal atau ilegal di dalam kode sumber. Selain itu, skenario ini juga dirancang untuk memastikan bahwa sistem kompilator dapat melaporkan lokasi spesifik dari kesalahan tersebut, baik baris maupun kolomnya, secara tepat kepada pengguna.

Berikut adalah kode sumber dengan kesalahan yang disengaja (penggunaan karakter @) untuk pengujian ini:

```
boolean a = true;
boolean b = @false;
print(a);
```

Hasil: Scanner berhenti dan melaporkan error berikut:

```
[Baris 2, Kolom 13] Error Leksikal: Karakter tidak dikenal: '@'
2 | boolean b = @false;
  |               ^
```

Gambar 11 Pesan error leksikal beserta penunjuk posisi

Tabel 19 Hasil Penelusuran Fase Kompilasi dan Optimasi Kode Pada Skenario Error Leksikal

=====
TEST CASE 2 – Error Leksikal
=====
Source Code:
boolean a = true;
boolean b = @false;
print(a);

[Baris 2, Kolom 13] Error Leksikal: Karakter tidak dikenal: '@'

```
[*] Scanning : Berhasil (16 token dihasilkan)
[*] Parsing  : Berhasil (AST terbentuk)
[*] Semantik : Berhasil (Variabel tervalidasi)
[*] ICG      : Berhasil (5 instruksi TAC mentah)

--- TAC Sebelum Optimasi ---

Three Address Code (TAC)

0:      t0 = true
1:      a = t0
2:      t1 = false
3:      b = t1
4:      print a

Total instruksi: 5

[*] Optimasi : Berhasil (3 instruksi setelah optimasi)

--- TAC Sesudah Optimasi ---

Three Address Code (TAC)

0:      a = true
1:      b = false
2:      print a

Total instruksi: 3

Laporan Optimasi:

Pass dijalankan          : 2
Instruksi di-fold        : 2
Instruksi dihapus (DCE) : 2
Penghematan              : 2 instruksi (40%)
```

Hasil Pengujian: Berdasarkan eksekusi yang dilakukan, modul *Scanner* langsung berhenti beroperasi ketika menemukan karakter yang tidak valid dan membatalkan proses kompilasi. Sistem kemudian memunculkan laporan *error* leksikal yang sangat informatif, lengkap dengan indikator penunjuk posisi kesalahan seperti berikut:

4.3 Test Case 3 – Error Sintaks (Multi Error)

Pengujian ini bertujuan untuk memverifikasi fungsionalitas mekanisme *panic-mode recovery* yang tertanam pada modul *Parser*. Skenario ini dirancang guna membuktikan bahwa sistem mampu mendeteksi dan melaporkan lebih dari satu kesalahan sintaksis secara beruntun dalam satu kali proses eksekusi, alih-alih langsung berhenti beroperasi saat menemukan *error* pertama.

Berikut adalah kode sumber yang disisipkan beberapa kesalahan tata bahasa secara disengaja untuk keperluan pengujian ini:

```
boolean a = true
boolean b = false;
if a AND b {
    print(a);
}
```

Gambar 12 Kode Sumber Input Test Case dengan Beberapa Kesalahan Teknis

Tabel 20 Riwayat Pengujian Skenario Negative Testing dengan Panic-mode Recovery pada Parser

```
=====
TEST CASE 3 – Error Sintaks (Multi Error)
=====

Source Code:
boolean a = true
boolean b = false;
if a AND b {
    print(a);
}

-----
[*] Scanning : Berhasil (21 token dihasilkan)
[Baris 2, Kolom 1] Error Sintaks: ';' diperlukan di akhir
deklarasi – ditemukan 'boolean' (BOOLEAN)
[Baris 3, Kolom 4] Error Sintaks: '(' diperlukan setelah 'if' –
ditemukan 'a' (IDENTIFIER)
[Baris 5, Kolom 1] Error Sintaks: Pernyataan tidak valid dimulai
dengan '}'

[!] Parsing : Gagal – Ditemukan Error Sintaksis.
-> [Baris 2, Kolom 1] Error Sintaks: ';' diperlukan di akhir
deklarasi – ditemukan 'boolean' (BOOLEAN)
-> [Baris 3, Kolom 4] Error Sintaks: '(' diperlukan setelah
'if' – ditemukan 'a' (IDENTIFIER)
-> [Baris 5, Kolom 1] Error Sintaks: Pernyataan tidak valid
dimulai dengan '}'
```

4.4 Test Case 4 Error Semantik

Pengujian ini bertujuan untuk memverifikasi keandalan modul *Semantic Analyzer* dalam mendeteksi pelanggaran aturan logika program. Skenario ini secara khusus difokuskan pada pengujian deteksi deklarasi ganda pada variabel yang sama, serta penggunaan variabel yang belum pernah dideklarasikan sebelumnya di dalam kode sumber.

Berikut adalah kode sumber yang disisipkan kesalahan logika secara disengaja untuk pengujian ini:

```

boolean a = true;
boolean a = false;
if (a AND z) {
    print(z);
}

```

Gambar 13 Test Case 4 Error Semantik

Tabel 21 Hasil Pengujian Skenario Negative Testing Untuk Validasi Variabel pada Semantik Analyzer

<pre> ===== TEST CASE 4 – Error Semantik ===== Source Code: boolean a = true; boolean a = false; if (a AND z) { print(z); } ----- [*] Scanning : Berhasil (24 token dihasilkan) [*] Parsing : Berhasil (AST terbentuk) [Baris 2] Error Semantik: Variabel 'a' sudah dideklarasikan di baris 1 [Baris 3] Error Semantik: Variabel 'z' belum dideklarasikan [Baris 4] Error Semantik: Variabel 'z' belum dideklarasikan [!] Semantik : Gagal – Ditemukan Error Semantik. </pre>
--

Hasil Pengujian: Berdasarkan hasil eksekusi, modul *Semantic Analyzer* berhasil mendeteksi dan melaporkan penemuan 3 buah *error* semantik secara akurat. Kesalahan yang ditangkap meliputi percobaan deklarasi ulang pada variabel *a* yang sudah terdaftar sebelumnya (terdeteksi di baris 2), serta penggunaan variabel *z* yang sama sekali belum pernah dideklarasikan (terdeteksi pemanggilannya di baris 3 dan 4).

4.5 Test Case 5 – Ekspresi Kompleks & Optimasi

Pengujian kelima ini ditujukan untuk memverifikasi kemampuan kompilator dalam menangani evaluasi ekspresi boolean bersarang yang melibatkan kombinasi kompleks antara operator AND, OR, dan NOT. Selain itu, skenario ini secara khusus dirancang untuk mengukur dan membuktikan tingkat efektivitas dari teknik optimasi yang telah ditanamkan pada sistem.

Berikut adalah kode sumber dengan logika percabangan dan ekspresi yang kompleks untuk keperluan pengujian ini:

```

boolean p = true;
boolean q = false;
boolean r = p OR q AND NOT p;
if ((p OR q) AND NOT r) {
    print(p);
} else {
    print(r);
}

```

Gambar 14 Hasil pengujian Test Case 5

Tabel 22 Log Statistik Perbandingan Komparatif Jumlah Instruksi Sebelumnya dan Sesudah Optimasi

Metrik	Nilai
Instruksi TAC sebelum optimasi	41 instruksi
Instruksi TAC sesudah optimasi	34 instruksi
Pass optimasi dijalankan	2 pass
Status kompilasi	Berhasil, tanpa error

Tabel 23 Hasil Pengujian dan Optimasi Kode Antar (TAC) Pada Skenario Ekspresi Logika Bersarang

<pre> ===== TEST CASE 5 – Ekspresi Kompleks & Optimasi ===== Source Code: boolean p = true; boolean q = false; boolean r = p OR q AND NOT p; if ((p OR q) AND NOT r) { print(p); } else { print(r); } ----- [*] Scanning : Berhasil (47 token dihasilkan) [*] Parsing : Berhasil (AST terbentuk) [*] Semantik : Berhasil (Variabel tervalidasi) [*] ICG : Berhasil (41 instruksi TAC mentah) --- TAC Sebelum Optimasi --- </pre>	
Three Address Code (TAC)	
0:	t0 = true
1:	p = t0
2:	t1 = false
3:	q = t1

```

4:      if p goto L0
5:      ifFalse q goto L2
6:      t4 = NOT p
7:      ifFalse t4 goto L2
8:      t3 = true
9:      goto L3
10: L2:
11:      t3 = false
12: L3:
13:      if t3 goto L0
14:      t2 = false
15:      goto L1
16: L0:
17:      t2 = true
18: L1:
19:      r = t2
20:      if p goto L6
21:      if q goto L6
22:      t6 = false
23:      goto L7
24: L6:
25:      t6 = true
26: L7:
27:      ifFalse t6 goto L4
28:      t7 = NOT r
29:      ifFalse t7 goto L4
30:      t5 = true
31:      goto L5
32: L4:
33:      t5 = false
34: L5:
35:      ifFalse t5 goto L8
36:      print p
37:      goto L9
38: L8:
39:      print r
40: L9:

```

Total instruksi: 41

[*] Optimasi : Berhasil (34 instruksi setelah optimasi)

--- TAC Sesudah Optimasi ---

Three Address Code (TAC)

```

0:      p = true
1:      q = false
2:      goto L0
3: L2:
4:      t3 = false
5: L3:
6:      if t3 goto L0
7:      t2 = false

```



```

8:      goto L1
9:  L0:
10:     t2 = true
11:  L1:
12:     r = t2
13:     if p goto L6
14:     if q goto L6
15:     t6 = false
16:     goto L7
17:  L6:
18:     t6 = true
19:  L7:
20:     ifFalse t6 goto L4
21:     t7 = NOT r
22:     ifFalse t7 goto L4
23:     t5 = true
24:     goto L5
25:  L4:
26:     t5 = false
27:  L5:
28:     ifFalse t5 goto L8
29:     print p
30:     goto L9
31:  L8:
32:     print r
33:  L9:

```

Total instruksi: 34

Laporan Optimasi:

Pass dijalankan	: 2
Instruksi di-fold	: 6
Instruksi dihapus (DCE):	7
Penghematan	: 7 instruksi (17%)

4.6 Ringkasan Hasil Pengujian

Berdasarkan serangkaian skenario pengujian yang telah dilakukan, kompilator SimpleLog terbukti mampu menangani berbagai kondisi eksekusi secara komprehensif, mulai dari pemrosesan program fungsional yang valid hingga pendeteksian berbagai tingkat kesalahan secara akurat. Secara keseluruhan, kelima skenario pengujian utama yang dieksekusi berhasil memenuhi target dan memberikan hasil yang sepenuhnya selaras dengan spesifikasi sistem yang dirancang.

Berikut adalah tabel ringkasan dari seluruh hasil pengujian yang telah dilaksanakan:

Tabel 24 Ringkasan Seluruh Hasil Pengujian (5/5 Lolos)

No	Test Case	Kategori	Status
1	Program valid dasar	Fungsional	Lolos
2	Karakter tidak dikenal	Error Leksikal	Lolos
3	Titik koma & kurung hilang	Error Sintaks	Lolos
4	Variabel ganda & tidak dideklarasikan	Error Semantik	Lolos
5	Ekspresi kompleks + optimasi	Fungsional & Optimasi	Lolos

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan serangkaian tahapan perancangan, implementasi, dan pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan utama dari proyek ini. Pertama, kompiler SimpleLog berhasil diimplementasikan secara utuh dengan keenam komponen wajib, yaitu *Scanner*, *Parser*, *Semantic Analyzer*, ICG, Optimasi, dan *Error Handling*, yang semuanya berfungsi dengan baik sesuai dengan spesifikasi tugas. Pendekatan *recursive descent parsing* yang digunakan terbukti sangat efektif dalam mengimplementasikan tata bahasa (*grammar*) SimpleLog yang berlapis. Melalui arsitektur penguraian ini, hierarki prioritas operator logika (NOT > AND > OR) dapat tertanam secara langsung di dalam struktur kode tanpa memerlukan tabel prioritas tambahan.

Selanjutnya, penerapan dua teknik optimasi—yakni *constant folding* dan *dead code elimination*—yang dijalankan secara iteratif terbukti berhasil mereduksi jumlah instruksi *Three Address Code* (TAC) secara signifikan. Pengujian menunjukkan adanya penghematan instruksi yang berkisar antara 17% hingga 28% pada berbagai skenario pengujian. Selama proses pengembangan, terungkap pula sebuah temuan teknis mengenai pentingnya kehati-hatian saat menerapkan *constant folding* pada kode yang memiliki struktur percabangan. Nilai konstanta yang telah diketahui kebenarannya pada satu cabang eksekusi tidak dapat diasumsikan berlaku sama pada titik temu (*label*) dari cabang lainnya. Temuan teknis ini memberikan pembelajaran yang sangat berharga mengenai urgensi analisis aliran kontrol (*control flow analysis*) dalam proses optimasi sebuah kompilator.

Terakhir, mekanisme *error handling* yang terintegrasi langsung pada setiap fase pemrosesan (leksikal, sintaksis, dan semantik) telah berhasil menyajikan pesan kesalahan yang sangat informatif. Pesan tersebut secara akurat memuat informasi mengenai kategori *error*, letak detail nomor baris dan kolom, serta cuplikan potongan kode sumber terkait, yang terbukti sangat memudahkan pengguna dalam proses *debugging* program SimpleLog.

5.2 Saran

Menyadari berbagai batasan dari ruang lingkup proyek kompilator ini, terdapat beberapa area potensial yang dapat menjadi fokus saran pengembangan lanjutan apabila proyek ini diteruskan. Saran pengembangan tersebut meliputi:

1. **Pengembangan Code Generator Akhir:** Menambahkan modul *code generator* (sebagai komponen bonus) yang mampu menerjemahkan TAC hasil optimasi ke dalam bahasa pemrograman tingkat tinggi seperti Python atau JavaScript, sehingga kode program SimpleLog dapat langsung dieksekusi.
2. **Perluasan Sistem Tipe Data:** Menambahkan dukungan terhadap sistem tipe data tambahan di luar boolean, seperti *integer* atau *string*, guna memperluas fungsionalitas dan cakupan penggunaan bahasa SimpleLog dalam komputasi matematis.

3. **Implementasi Optimasi Lanjutan:** Mengimplementasikan teknik optimasi kompilator tingkat lanjut, seperti *common subexpression elimination*, yang berguna untuk menyederhanakan dan mengeliminasi proses evaluasi ekspresi yang dilakukan secara berulang.
4. **Integrasi Unit Testing Otomatis:** Menambahkan ekosistem pengujian unit (*unit test*) secara otomatis menggunakan *framework* terstandarisasi seperti pytest. Hal ini sangat vital diimplementasikan agar setiap regresi atau kerusakan teknis pada salah satu modul dapat terdeteksi secara otomatis dan lebih cepat ketika siklus pengembangan kode berlanjut.

DAFTAR PUSTAKA

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Education.

Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J. H., & Langendoen, K. (2012). *Modern Compiler Design* (2nd ed.). Springer.

Louden, K. C. (1997). *Compiler Construction: Principles and Practice*. Cengage Learning.

Nystrom, R. (2021). *Crafting Interpreters*. Genever Benning.

Python Software Foundation. (2026). *The Python 3.10 Language Reference*. Diakses pada Juni 2026, dari <https://docs.python.org/3.10/reference/>

Sebesta, R. W. (2012). *Concepts of Programming Languages* (10th ed.). Pearson.